

REACT ROUTER (v6) - COMPLETE NOTES Page

1. What is React Router ?

- React Router is the standard routing library for React.
- It helps us to build Single Page Applications (SPA) with multiple views without page reload.
- It keeps the UI in sync with the URL.

Navigate
Without
Reloading
the page
♥

2. Why React Router ?

- ✓ Better User Experience (no page reload)
- ✓ Bookmarkable URLs
- ✓ Protect Routes (Private / Public)
- ✓ Dynamic Routing
- ✓ Code Splitting & Lazy Loading
- ✓ Nested Routes

3. Installation

```
npm install react-router-dom
```

We use
react-router-dom
(not just
react-router)
in v6

4. Basic Setup

```
// main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

BrowserRouter
wraps the
entire app
(only once)

5. Routes & Route (Basic)

```
// App.jsx
import { Routes, Route } from 'react-router-dom'
import Home from './pages/Home'
import About from './pages/About'
import Contact from './pages/Contact'

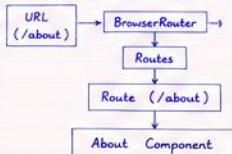
function App() {
  return (
    <Routes>
      <Route path="/" element={ <Home /> } />
      <Route path="/about" element={ <About /> } />
      <Route path="/contact" element={ <Contact /> } />
    </Routes>
  )
}
```

In v6
element={ }
is used
instead of
component=

Note:

- Only one <Route> is rendered at a time (best match).
- <Routes> must wrap all <Route>.

How it Works ?



6. Navigation (Link & NavLink)

<u>Link (Simple)</u>	<u>NavLink (With Active Style)</u>
<pre>import { Link } from 'react-router-dom' <Link to="/">Home</Link> <Link to="/about">About</Link> → Used for simple navigation.</pre>	<pre>import { NavLink } from 'react-router-dom' <NavLink to="/about" className={({ isActive }) => isActive ? 'active-link' : 'link' > About </NavLink> → Useful to show active link.</pre>

CSS Example:

```
.link {
  color: blue;
  text-decoration: none;
  margin: 0 10px;
}

.active-link {
  color: red;
  font-weight: bold;
  border-bottom: 2px solid red;
}
```

7. useNavigate() Hook (Programmatic Navigation)

```
import { useNavigate } from 'react-router-dom'
function Login() {
  const navigate = useNavigate();
  const handleLogin = () => {
    navigate('/dashboard'); // redirect after login
  }
  return ( <button onClick={handleLogin}>Login </button> )
}
```

useNavigate()
replaces
useHistory()
in v6

8. useParams() Hook (Dynamic Routes)

```
import { useParams } from 'react-router-dom'
function User() {
  const { id } = useParams(); // get :id from URL
  return ( <h2>User ID: {id}</h2> )
}
```

Route Example:

```
<Route
  path="/user/:id"
  element=<User />
 />
```

9. useLocation() Hook

```
import { useLocation } from 'react-router-dom'
function ShowLocation() {
  const location = useLocation();
  return (
    <div>
      <p>Pathname: {location.pathname}</p>
      <p>Search: {location.search}</p>
      <p>Hash: {location.hash}</p>
    </div>
  )
}
```

Useful for
getting current
URL info
(pathname,
search, hash)

Common Hooks Summary

useNavigate()	→ For navigation
useParams()	→ For dynamic route params
useLocation()	→ For current URL info
useMatch()	→ To match a path
useSearchParams()	→ To read/set query params

Example URL

/user/42?name=kapil#top

pathname → /user/42

search → ?name=kapil

hash → #top

10. Nested Routes

```

<Route path="/" element={<Layout />}>
  <Route index element={<Home />} />  {/* default child */}
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Route>

```

```

// In Layout.jsx
import { Outlet } from 'react-router-dom'
function Layout() {
  return (
    <div>
      <Navbar />
      <Outlet />      {/* child routes will render here */}
      <Footer />
    </div>
  )
}

```

<Outlet />
is used to
render child
routes

11. Protected / Private Route (Simple Way)

```

function PrivateRoute({ children }) {
  const isLoggedIn = localStorage.getItem('token');
  if (!isLoggedIn) {
    return <Navigate to="/login" replace />
  }
  return children;
}

<Route path="/dashboard" element={
  <PrivateRoute>
    <Dashboard />
  </PrivateRoute>
} />

```

Navigate

- Used to redirect
- replace → prevents going back to previous page

12. Lazy Loading (Code Splitting)

```

const Home = lazy(() => import('../pages/Home'))
const About = lazy(() => import('../pages/About'))

<Suspense fallback={<h2>Loading...</h2>}>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</Suspense>

```

Improves
performance
by loading
components
on demand

Quick Revision

- ✓ React Router helps in navigation without page reload.
- ✓ Use <BrowserRouter> at the root.
- ✓ Define routes using <Routes> & <Route>.
- ✓ Use Link / NavLink for navigation.
- ✓ useNavigate() for programmatic navigation.
- ✓ useParams() for dynamic params.
- ✓ useLocation() for current URL info.
- ✓ Nested routes use <Outlet />.
- ✓ Protect routes using <PrivateRoute> + <Navigate>.
- ✓ Lazy loading improves performance.

Complete Flow